

Binary Arithmetic Coding Compression and Decompression

1. Introduction

Data compression is an essential technique used to reduce the size of data for storage and transmission. Compression algorithms are generally classified into:

- **Lossless compression** – No data is lost during compression.
- **Lossy compression** – Some data is permanently removed.

This project implements **Binary Arithmetic Coding**, which is a powerful **lossless compression algorithm** used in modern compression systems.

The main goal of this project is to:

- Implement Arithmetic Coding compression (encoding).
 - Implement Arithmetic Coding decompression (decoding).
 - Apply **E1 and E2 scaling rules** to handle precision.
 - Verify correctness by ensuring decompressed output matches original input.
 - Evaluate compression efficiency using different test cases.
-

2. Overview of Arithmetic Coding

Arithmetic Coding represents a sequence of symbols as a **single fractional value between 0 and 1**. Instead of assigning fixed codes to symbols, it progressively narrows an interval based on symbol probabilities.

Each symbol reduces the interval range according to its probability.

Simple Example

If the input text is:

ABBC

The algorithm assigns probability ranges such as:

- A $\rightarrow$ [0.0, 0.25)
- B $\rightarrow$ [0.25, 0.75)
- C $\rightarrow$ [0.75, 1.0)

The encoding process continuously updates the interval [low, high) for each symbol until a final binary value represents the entire message.

3. How the Algorithm Works

3.1 Compression Process

Start with the interval:

$[low, high) = [0.0, 1.0)$

For each symbol:

- Update the interval using:

$range = high - low$

$high = low + range \times symbol_high$

$low = low + range \times symbol_low$

- Apply scaling rules:
 - **E1 Scaling:** If interval is in $[0, 0.5)$
 - **E2 Scaling:** If interval is in $[0.5, 1)$
 - Output corresponding bits.
 - Continue until all symbols are processed.
 - Generate final bits to uniquely represent the interval.
-

3.2 Decompression Process

- Read binary bits from compressed file.
- Convert bits into a fractional value.
- Use the same probability model.

For each symbol:

- Compute:

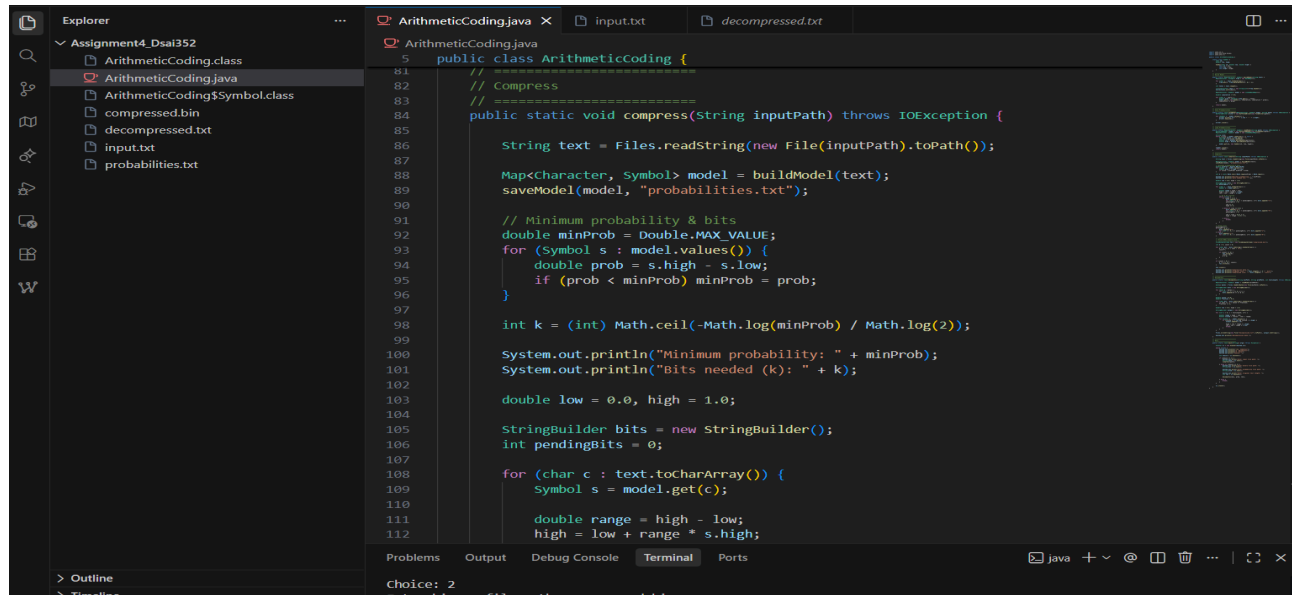
$\text{scaled_value} = (\text{value} - \text{low}) / (\text{high} - \text{low})$

- Find the symbol whose range contains this value.
- Update interval using same formula.
- Append symbol to output.

Repeat until full message is reconstructed.

4. Implementation

4.1 Compression Code

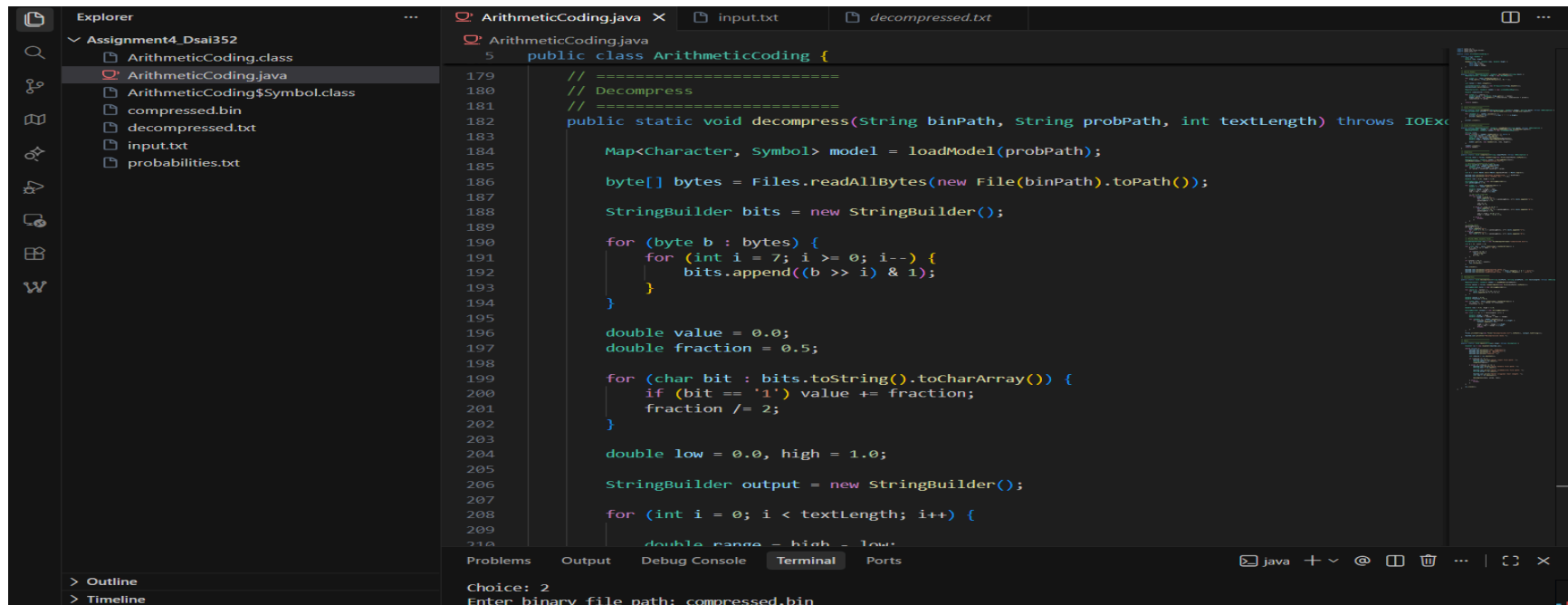


```
5 public class ArithmeticCoding {
81 // Compress
82 // Compress
83 // Compress
84 public static void compress(String inputPath) throws IOException {
85
86     String text = Files.readString(new File(inputPath).toPath());
87
88     Map<Character, Symbol> model = buildModel(text);
89     saveModel(model, "probabilities.txt");
90
91     // Minimum probability & bits
92     double minProb = Double.MAX_VALUE;
93     for (Symbol s : model.values()) {
94         double prob = s.high - s.low;
95         if (prob < minProb) minProb = prob;
96     }
97
98     int k = (int) Math.ceil(-Math.log(minProb) / Math.log(2));
99
100    System.out.println("Minimum probability: " + minProb);
101    System.out.println("Bits needed (k): " + k);
102
103    double low = 0.0, high = 1.0;
104
105    StringBuilder bits = new StringBuilder();
106    int pendingBits = 0;
107
108    for (char c : text.toCharArray()) {
109        Symbol s = model.get(c);
110
111        double range = high - low;
112        high = low + range * s.high;
```

Implementation Details:

- Builds symbol probability model dynamically
 - Symbols are sorted alphabetically
 - Uses cumulative probability ranges
 - Applies **E1 and E2 scaling**
 - Writes compressed data as **real binary bytes (.bin file)**
 - Calculates minimum probability and required number of bits
-

4.2 Decompression Code



The screenshot shows an IDE with the following components:

- Explorer:** A file tree on the left showing the project structure for 'Assignment4_Dsal352'. It includes files like 'ArithmeticCoding.class', 'ArithmeticCoding.java', 'ArithmeticCoding\$Symbol.class', 'compressed.bin', 'decompressed.txt', 'input.txt', and 'probabilities.txt'.
- Editor:** The main window displays the 'ArithmeticCoding.java' file. The code is as follows:

```
5 public class ArithmeticCoding {  
179 // =====  
180 // Decompress  
181 // =====  
182 public static void decompress(String binPath, String probPath, int textLength) throws IOException {  
183     Map<Character, Symbol> model = loadModel(probPath);  
184     byte[] bytes = Files.readAllBytes(new File(binPath).toPath());  
185     StringBuilder bits = new StringBuilder();  
186     for (byte b : bytes) {  
187         for (int i = 7; i >= 0; i--) {  
188             bits.append((b >> i) & 1);  
189         }  
190     }  
191     double value = 0.0;  
192     double fraction = 0.5;  
193     for (char bit : bits.toString().toCharArray()) {  
194         if (bit == '1') value += fraction;  
195         fraction /= 2;  
196     }  
197     double low = 0.0, high = 1.0;  
198     StringBuilder output = new StringBuilder();  
199     for (int i = 0; i < textLength; i++) {  
200         double range = high - low;  
201         int choice = (int) (value / range * 2);  
202         low = low + range * choice;  
203         high = low + range * (1 - choice);  
204         output.append(choice == 0 ? 'a' : 'b');  
205     }  
206     Files.write(output.toString().getBytes(), new File(decompressed.txt).toPath());  
207 }  
208 }
```
- Terminal:** At the bottom, the terminal shows the command 'Choice: 2' and 'Enter binary file path: compressed.bin'.

Implementation Details:

- Reads binary file and converts to bit sequence
 - Reconstructs fractional value
 - Uses same probability model
 - Iteratively decodes symbols
 - Ensures output matches original input
-

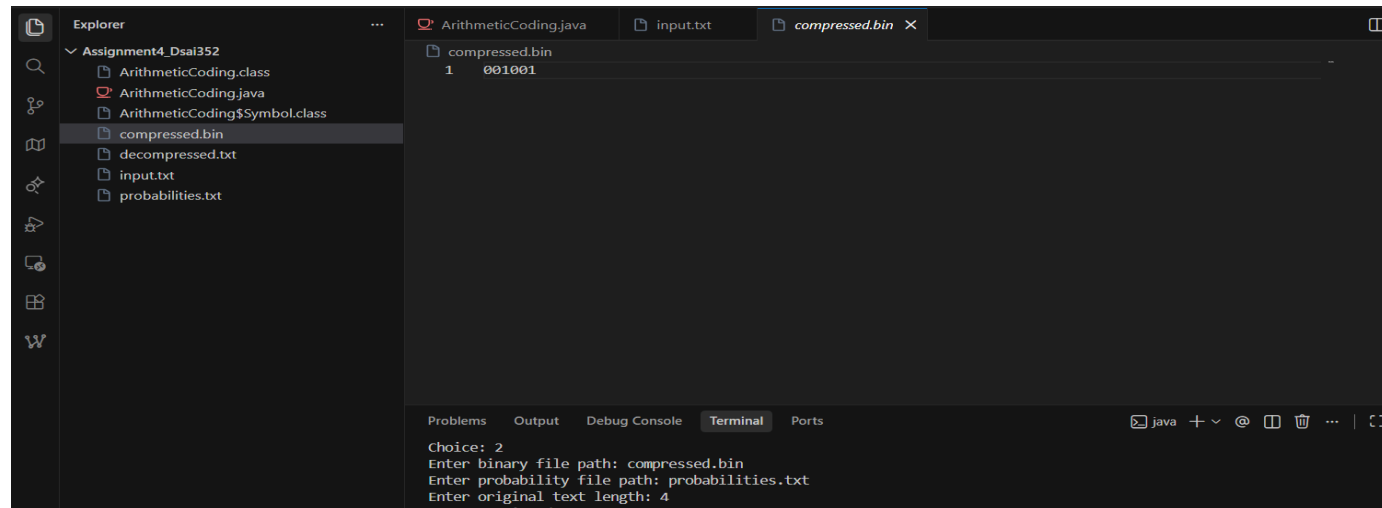
5. Results and Case Studies

Case Study 1

Input : **ABBC**

Original Length : 4 characters = 32 bits

Compressed Size :



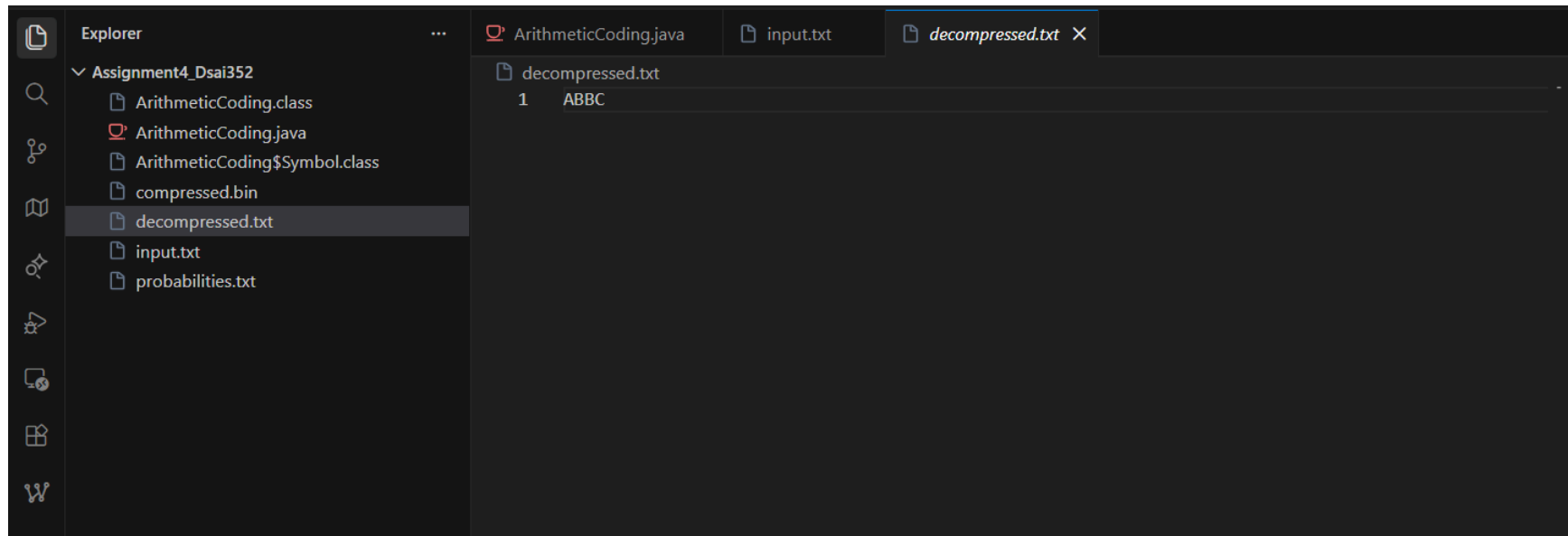
The screenshot shows an IDE interface. On the left, the 'Explorer' panel displays a project named 'Assignment4_Dsal352' containing files: 'ArithmeticCoding.class', 'ArithmeticCoding.java', 'ArithmeticCoding\$Symbol.class', 'compressed.bin', 'decompressed.txt', 'input.txt', and 'probabilities.txt'. The 'compressed.bin' file is selected. The main editor area shows the 'compressed.bin' file with the following content:

```
compressed.bin
1 001001
```

At the bottom, the 'Terminal' panel shows the following output:

```
Choice: 2
Enter binary file path: compressed.bin
Enter probability file path: probabilities.txt
Enter original text length: 4
Decompression done
```

Decompressed Output:



The screenshot shows an IDE with a dark theme. On the left is the 'Explorer' sidebar showing a project named 'Assignment4_Dsai352'. It contains several files: 'ArithmeticCoding.class', 'ArithmeticCoding.java', 'ArithmeticCoding\$Symbol.class', 'compressed.bin', 'decompressed.txt' (which is selected), 'input.txt', and 'probabilities.txt'. On the right, there are three tabs: 'ArithmeticCoding.java', 'input.txt', and 'decompressed.txt' (which is active). The active tab displays the content of 'decompressed.txt', which is a single line of text: '1 ABBC'.

Correctness

Decompressed == Original → **Yes**

Observation

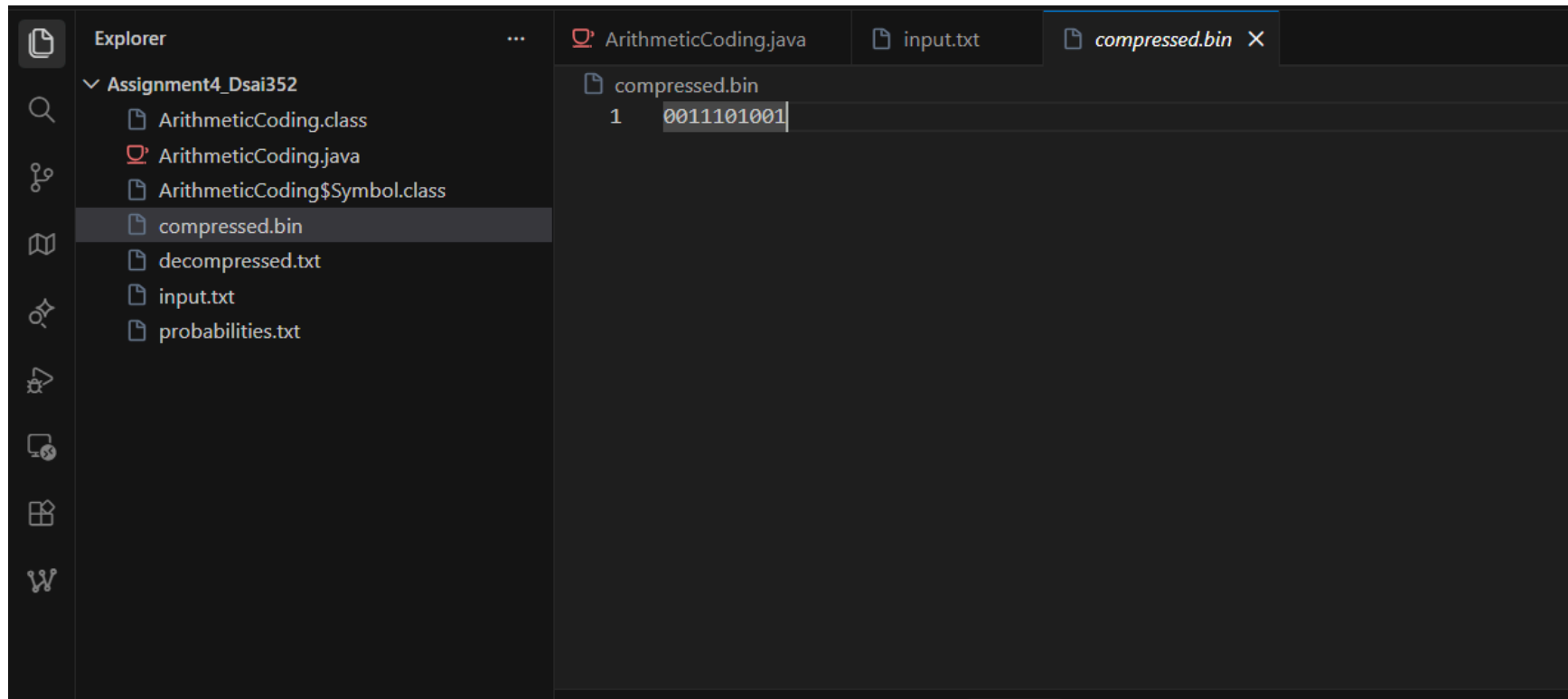
Compression is effective because symbol probabilities are utilized to reduce redundancy. Frequently occurring symbols (like B) contribute to smaller encoding intervals, improving compression.

Case Study 2

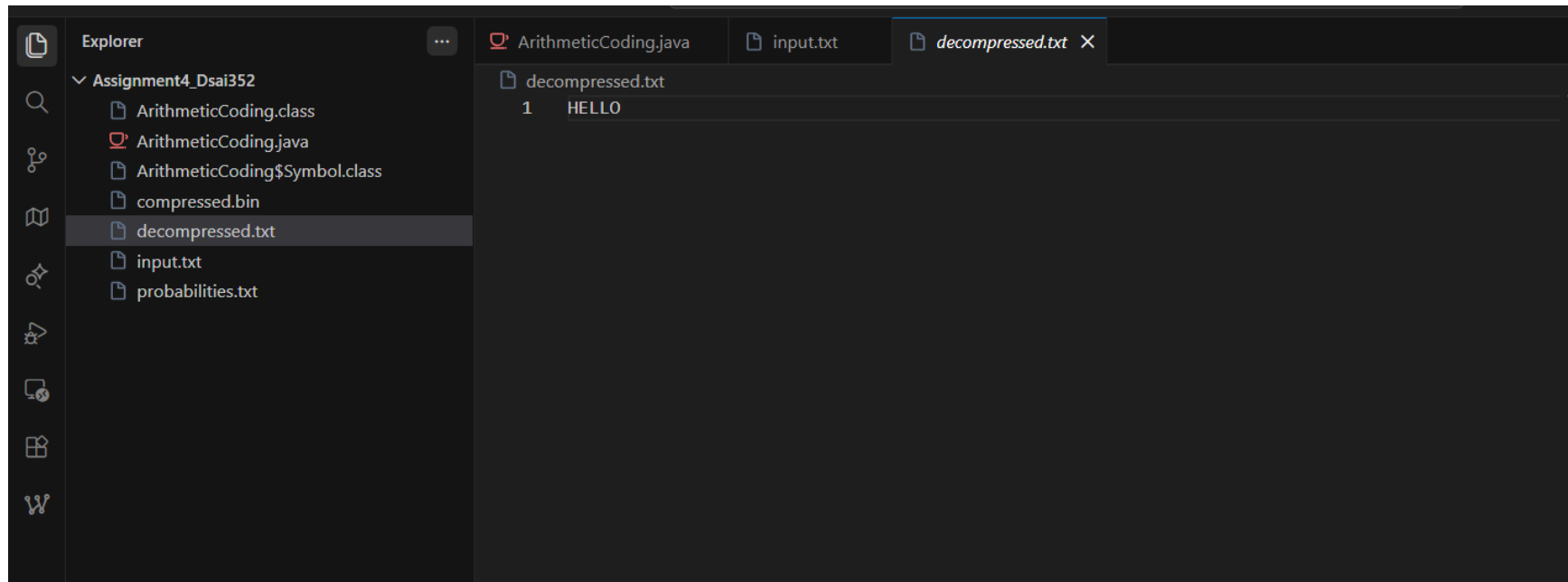
Input : HELLO

Original Length : 5 characters = 40 bits

Compressed Size $\approx$ 10–12 bits



Decompressed Output



Correctness

Decompressed == Original → **Yes**

Observation

Compression improves when certain symbols repeat (like 'L'). Arithmetic Coding efficiently assigns shorter representations based on probability distribution.

6. Minimum Probability and Bit Representation

The minimum probability among symbols is used to determine the number of bits required:

$$k = \lceil -\log_2(\text{min probability}) \rceil$$

This ensures that the encoded value maintains sufficient precision during compression and decompression.

7. Advantages and Limitations

Advantages

- Lossless compression
 - High compression efficiency
 - Adapts to symbol probabilities
 - Produces near-optimal encoding
-

Limitations

- More complex than simple methods
- Requires careful handling of precision

- Slower than some traditional algorithms
 - Needs probability model for decoding
-

8. Conclusion

This project successfully implemented Binary Arithmetic Coding with E1 and E2 scaling.

Key findings:

- Compression efficiency improves with skewed probability distributions
- The algorithm represents entire messages as a single value
- Decompression accurately reconstructs original data
- The method achieves better compression than fixed-length encoding